



Reverse a LinkedList (easy)

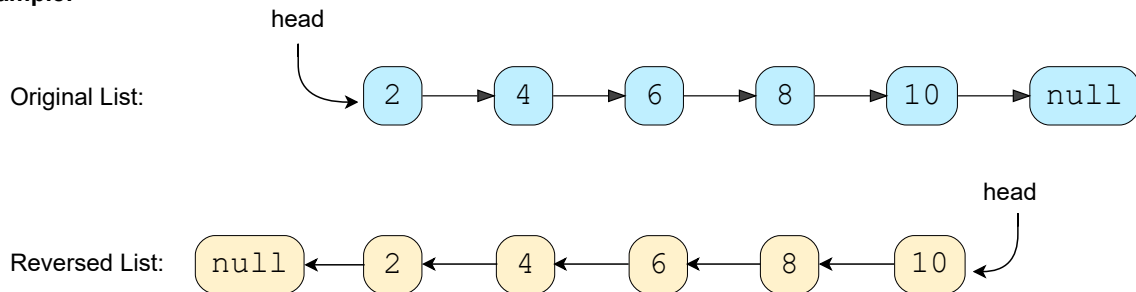
We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
- Time complexity
- Space complexity

Problem Statement

Given the head of a Singly LinkedList, reverse the LinkedList. Write a function to return the new head of the reversed LinkedList.

Example:



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 class ListNode {
2     int value = 0;
3     ListNode next;
4
5     ListNode(int value) {
6         this.value = value;
7     }
8 }
9
10 class ReverseLinkedList {
```



```

11
12 public static ListNode reverse(ListNode head) {
13     // TODO: Write your code here
14     return head;
15 }
16
17 public static void main(String[] args) {
18     ListNode head = new ListNode(2);
19     head.next = new ListNode(4);
20     head.next.next = new ListNode(6);
21     head.next.next.next = new ListNode(8);
22     head.next.next.next.next = new ListNode(10);
23
24     ListNode result = ReverseLinkedList.reverse(head);
25     System.out.print("Nodes of the reversed LinkedList are: ");
26     while (result != null) {
27         System.out.print(result.value + " ");
28         result = result.next;
29     }
30 }
31 }

```

Run

Save

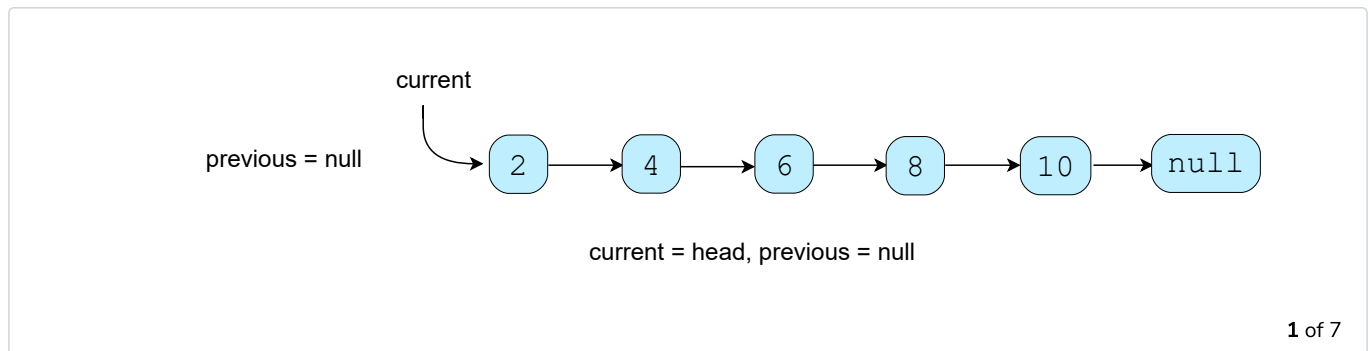
Reset



Solution

To reverse a LinkedList, we need to reverse one node at a time. We will start with a variable **current** which will initially point to the head of the LinkedList and a variable **previous** which will point to the previous node that we have processed; initially **previous** will point to **null**.

In a stepwise manner, we will reverse the **current** node by pointing it to the **previous** before moving on to the next node. Also, we will update the **previous** to always point to the previous node that we have processed. Here is the visual representation of our algorithm:



Code

Here is what our algorithm will look like:

Java	Python3	C++	JS
------	---------	-----	----

```

1 class ListNode {
2     int value = 0;
3     ...

```



```
3     ListNode next;
4
5     ListNode(int value) {
6         this.value = value;
7     }
8 }
9
10 class ReverseLinkedList {
11
12     public static ListNode reverse(ListNode head) {
13         ListNode current = head; // current node that we will be processing
14         ListNode previous = null; // previous node that we have processed
15         ListNode next = null; // will be used to temporarily store the next node
16
17         while (current != null) {
18             next = current.next; // temporarily store the next node
19             current.next = previous; // reverse the current node
20             previous = current; // before we move to the next node, point previous to the current node
21             current = next; // move on the next node
22         }
23         // after the loop current will be pointing to 'null' and 'previous' will be the new head
24         return previous;
25     }
26
27     public static void main(String[] args) {
28         ListNode head = new ListNode(2);
29         head.next = new ListNode(4);
30         head.next.next = new ListNode(6);
31         head.next.next.next = new ListNode(8);
32         head.next.next.next.next = new ListNode(10);
33
34         ListNode result = ReverseLinkedList.reverse(head);
35         System.out.print("Nodes of the reversed LinkedList are: ");
36         while (result != null) {
37             System.out.print(result.value + " ");
38             result = result.next;
39         }
40     }
41 }
```

Run

Save

Reset



Time complexity

The time complexity of our algorithm will be $O(N)$ where 'N' is the total number of nodes in the LinkedList.

Space complexity

We only used constant space, therefore, the space complexity of our algorithm is $O(1)$.

[← Back](#)[✓ Completed](#)[Next →](#)[Introduction](#)[Reverse a Sub-list \(medium\)](#)